

P.POORNA CHANDRA-192472287

Case Study 1 – Harris Corner Detection for Satellite Image Registration

Introduction

Satellite images of the same geographical region may be captured at different times under different illumination, orientations, and background conditions. Accurate image registration requires finding stable feature points that can be matched between images. Initially, edge detection is used, but it is not reliable enough for this application. Harris Corner Detection provides more distinctive and stable feature points for registration.

Limitations of Edge Detection

Edge detection identifies boundaries where image intensity changes significantly. However, it has several limitations in satellite image registration:

1. **Sensitivity to illumination:** Changes in sunlight, shadows, and atmospheric conditions can change edge strength and location.
2. **Orientation changes:** When the satellite images are captured from different viewing angles, the same edges may appear at different positions or directions.
3. **Background details:** Roads, buildings, vegetation, water bodies, and shadows can create many similar edges, making correct matching difficult.
4. **Edges are not highly distinctive:** An edge represents a boundary but does not provide a unique point for matching.
5. **Localization problem:** Small changes in image conditions can shift or weaken an edge, resulting in inaccurate alignment.
6. **Poor matching reliability:** Long edges may correspond to many possible locations in another image, increasing false matches.

Application of Harris Corner Detection

Harris Corner Detection identifies points where the image intensity changes significantly in **both horizontal and vertical directions**. These points are called corners.

The basic principle is based on the change in image intensity when a small window is shifted in different directions. A Harris response value is calculated as:

$$R = \det(M) - k(\text{trace}(M))^2$$

where **M** is the structure tensor and **k** is a sensitivity constant.

A large positive Harris response indicates a strong corner.

How Harris Improves Feature Localization

Corners are more precisely localized than general edges because they provide a specific intersection or distinctive point. For example, the intersection of two roads or the corner of a building can act as a stable reference point.

Therefore, instead of matching entire edges, the system can match specific feature points between the two satellite images.

Improvement in Matching Accuracy

Harris corners provide more distinctive locations for correspondence. A corner detected in one image can be compared with nearby corner features in the second image.

This reduces ambiguity because:

Edge → many possible matching locations

Corner → specific feature location

As a result, the number of incorrect correspondences can be reduced and the estimated transformation between images becomes more accurate.

Robustness Under Varying Imaging Conditions

Harris detection is more reliable than simple edge matching because it uses local intensity variations in two directions. Strong structural points such as building corners, road intersections, and other geographical landmarks can remain detectable even when the overall image appearance changes.

However, Harris detection is not completely invariant to large scale changes or significant rotation. Therefore, for very large viewpoint or scale variations, advanced detectors such as SIFT or other scale/rotation-invariant methods may provide better robustness.

Proposed Solution

The improved registration pipeline can be:

Satellite Images → Preprocessing → Harris Corner Detection → Feature Point Selection → Feature Matching → Transformation Estimation → Image Alignment

The system should select strong Harris corners and reject weak or unstable points. A suitable matching strategy and outlier rejection method such as RANSAC can then be used to estimate the transformation accurately.

CODE:

```
case1.py - C:/Users/POORNA SKHT/OneDrive/Tài liệu/computer vision ita0502/lab experime...
File Edit Format Run Options Window Help
import cv2
import numpy as np

# Load satellite image
img = cv2.imread("satellite.JPg")

if img is None:
    print("Image not found!")
    print("Check that the file name is satellite.JPg")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert to float32
gray = np.float32(gray)

# Harris Corner Detection
corners = cv2.cornerHarris(gray, 2, 3, 0.04)

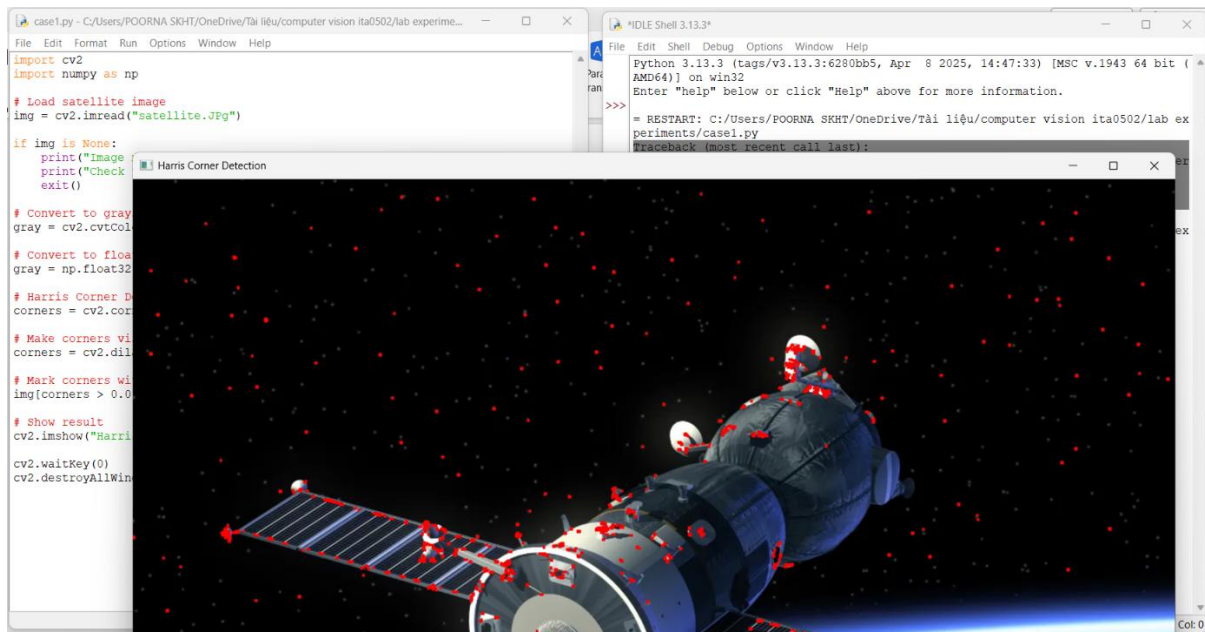
# Make corners visible
corners = cv2.dilate(corners, None)

# Mark corners with red color
img[corners > 0.01 * corners.max()] = [0, 0, 255]

# Show result
cv2.imshow("Harris Corner Detection", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



Conclusion

Edge detection is useful for identifying boundaries but is less suitable for accurate image registration because edges are not sufficiently distinctive and are sensitive to illumination, orientation, and background changes. Harris Corner Detection provides localized and distinctive feature points, improving correspondence, matching accuracy, and registration reliability. Therefore, Harris Corner Detection is a better choice for this case, particularly when the images contain stable structural features such as buildings, roads, and intersections.

Case Study 2 – Grayscale vs Binary vs Colour Images

Introduction

An autonomous surveillance system needs to process high-resolution images continuously to detect vehicles and pedestrians in real time. Processing full RGB color images requires more data and computational resources. Therefore, grayscale and binary representations can be considered to improve processing speed.

Comparison of Image Representations

Representation	Information	Computational Cost	Edge Detection	Feature Extraction
Color (RGB)	Red, green and blue information	High	Good	Rich but computationally expensive
Grayscale	Intensity information	Medium/Low	Very good	Good for most shape-based features
Binary	Only black/white information	Very low	Depends on thresholding	Limited

1. Color Image

RGB images contain three channels: red, green and blue. Therefore, a high-resolution color image contains approximately three times the pixel-channel data of a single-channel grayscale image.

Advantages:

- Preserves color information.
- Useful when color helps distinguish objects.
- Provides rich information for advanced object detection.

Limitations:

- Higher memory requirement.
- More computations during preprocessing and feature extraction.
- Can slow down real-time processing.

2. Grayscale Image

A grayscale image represents each pixel using intensity rather than three color channels. This significantly reduces the amount of data that needs to be processed.

Advantages:

- Lower computational complexity.

- Faster image processing.
- Good representation for edges, corners and shapes.
- Works well with feature detectors such as Harris and many traditional computer-vision algorithms.
- Preserves important structural information.

For example:

RGB → 3 channels

Grayscale → 1 channel

Therefore, grayscale processing can provide a good balance between computational efficiency and useful visual information.

3. Binary Image

A binary image contains only two values, normally **0 and 1** or **black and white**. It requires very little computational and storage capacity.

Advantages:

- Very fast processing.
- Simple representation.
- Useful for segmentation when foreground and background can be clearly separated.

Limitations:

- Loses most intensity information.
- Sensitive to threshold selection.
- Small changes in illumination can cause incorrect segmentation.
- Fine object details may disappear.
- Feature extraction becomes less reliable when important intensity variations are removed.

For surveillance scenes containing shadows, different lighting conditions, vehicles of different colors and pedestrians against complex backgrounds, binary representation can remove useful information.

Impact on Edge Detection

Edge detection generally depends on changes in image intensity.

Colour: Provides information from multiple channels but requires more computation.

Grayscale: Retains the major intensity changes needed to identify object boundaries while reducing computation. Therefore, it is highly suitable for traditional edge detectors such as Sobel and Canny.

Binary: Boundaries are already represented as transitions between black and white. However, incorrect thresholding can create missing edges or unwanted edges.

Hence, grayscale generally provides the best balance for reliable edge detection.

Impact on Feature Extraction

Feature detectors such as Harris Corner Detection and many traditional feature extraction methods mainly require intensity information rather than full RGB information.

Grayscale:

- Preserves important structural information.
- Allows faster feature extraction.
- Reduces memory and processing requirements.
- Maintains useful edges, corners and shapes.

Binary:

- Can make feature extraction faster.
- But may remove texture and intensity information needed for reliable feature detection.

Colour:

- Contains the most information.
- But processing all three channels increases computational cost.

Recommended Representation

For this real-time surveillance application, **grayscale is the most appropriate basic representation.**

The recommended pipeline is:

RGB Image → Grayscale Conversion → Noise Reduction → Edge/Feature Detection → Object Detection → Tracking

Grayscale provides a good compromise between **speed, accuracy and useful visual information.**

Binary representation should be used only when a reliable segmentation condition exists, such as a clearly separated foreground and background. Color information can be retained when color is essential for distinguishing objects.

CODE:

```
case2.py - C:/Users/POORNA SKHT/OneDrive/Tài liệu/computer vision ita0502/lab experime...
File Edit Format Run Options Window Help
import cv2

# Read color image
img = cv2.imread("satellite.jpg")

if img is None:
    print("Image not found!")
    print("Make sure satellite.jpg is in the same folder.")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert grayscale to binary
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

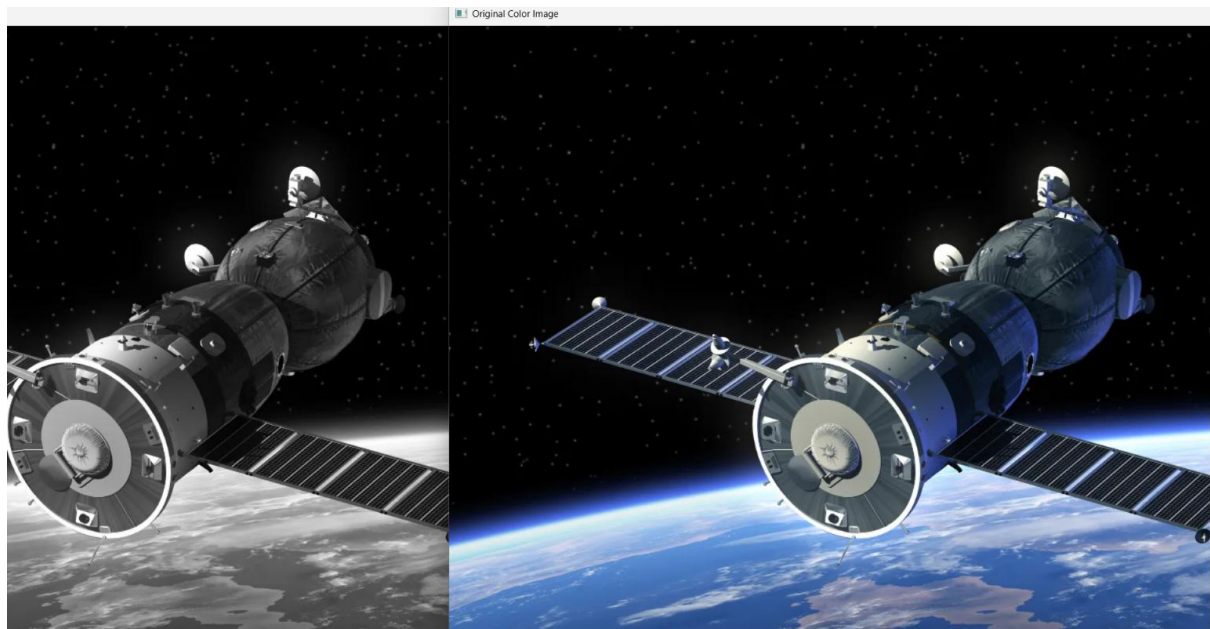
# Edge detection
edges = cv2.Canny(gray, 100, 200)

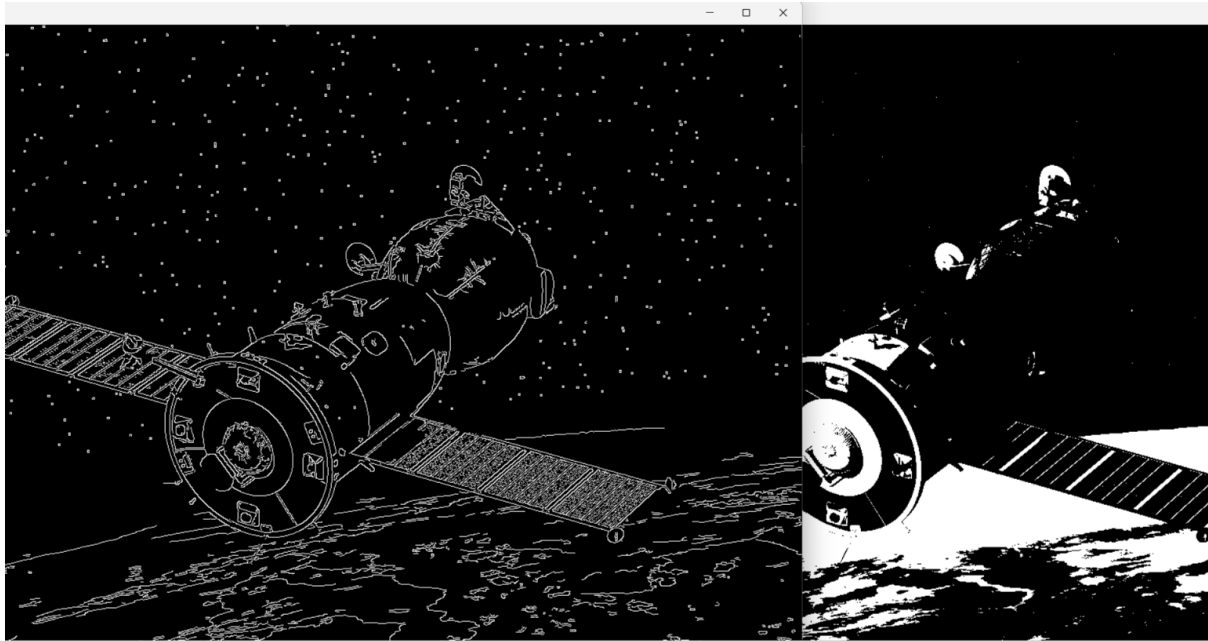
# Display images
cv2.imshow("Original Color Image", img)
cv2.imshow("Grayscale Image", gray)
cv2.imshow("Binary Image", binary)
cv2.imshow("Edge Detection", edges)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Ln: 8 Col: 35

OUTPUT:





Conclusion:

Although binary images provide the lowest computational cost, they discard too much useful information and are highly dependent on threshold selection. RGB images preserve maximum information but require greater computational resources.

Grayscale images reduce computational complexity while retaining the intensity information needed for edges, corners, shapes and many feature-extraction algorithms.

Therefore, **grayscale is the most suitable representation for accurate and reliable real-time object detection**, especially when combined with efficient feature detection and tracking methods.